



THE UNIVERSITY
OF QUEENSLAND
AUSTRALIA

INFS7450 SOCIAL MEDIA ANALYTICS (ST LUCIA) - SEMESTER 1, 2026

Project Two: Link Prediction

Francesco Balducci

Danmarks Tekniske Universitet

s250200@student.dtu.dk

s5010355@student.uq.edu.au

May 28, 2026

Contents

1 Introduction 1

2 Problem 2

3 Methods 3

3.1 Dataset split 3

3.2 Scoring method types 3

3.3 Method 1: Common Neighbors (neighborhood-based) 4

3.4 Method 2: Spectral Clustering (community-based) 4

3.5 Method 3: Graph Convolutional Network (embedding-based) 4

4 Results 6

4.1 Accuracy (method selection) 6

4.2 Predictions (method 2) 6

5 Discussion 8

5.1 Success areas 8

5.2 Improvement areas 8

6 Conclusion 9

A Appendix i

A.1 Sources i

A.2 Methods scoring on test set i

A.2.1 Method 1: Common Neighbors (3.3) i

A.2.2 Method 2: Spectral Clustering (3.4) i

A.2.3 Method 3: Graph Convolutional Network (3.5) ii

A.3 About Generative AI Usage iii

1 Introduction [link](#) · [index](#)

In the context of a graph structure, a range of tasks can be performed on nodes, edges, or the entire graph. These tasks include:

- **Classification:** e.g., identify spam nodes, detect fraudulent accounts
- **Prediction:** e.g., whether two nodes are likely to form a new link, predict which users will be reached from a given post
- **Community Detection:** e.g., identify echo chambers, uncover groups of coordinated bots
- **Similarity measurement:** e.g., estimate how much a group has homogeneous interests, recommend similar users

The complex structure of graphs necessitates us to navigate through these tasks in order to examine the network. Consider, for example, the following representations, all of which refer to the same graph at the center of this study:

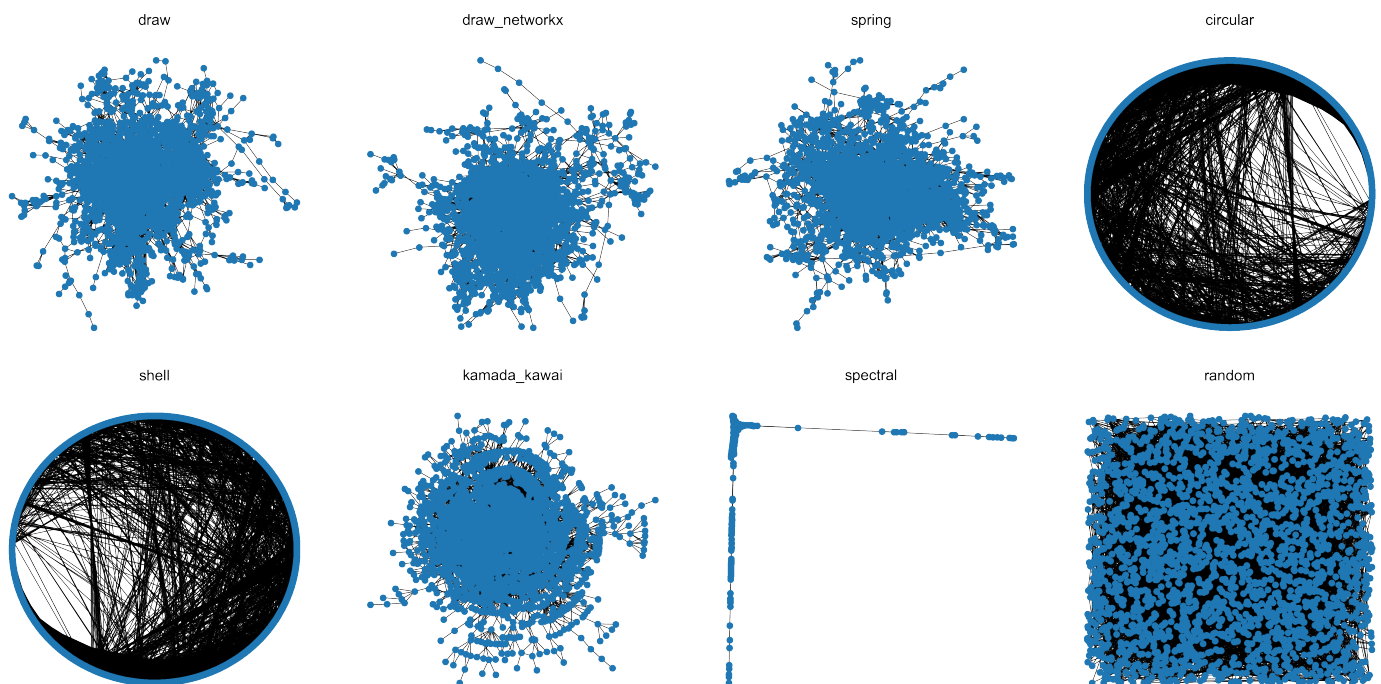


Figure 1: Different representations of the dataset related to the current study 2.

It is evident that the only way we have to comprehend the network is through the application of specific, detailed, and well-defined methods. In this study, the focus will be on examining the edge structure, with a particular focus on **predicting new edges**.

2 Problem [link](#) · [index](#)

The provided dataset is subdivided in two different files, `trainig.csv` and `testing.csv`. The first includes only *positive* edges, meaning edges that exist and are visible in Figure 1. The latter has a 1:9 ratio of **unlabeled** *positive* and *negative* edges. Our task is to analyze the known training structure in order to extract meaningful features and predict which edges from the test set are most likely to belong to the network.

The following table summarizes the composition of the dataset on which all of the methods 3 will be based:

	trainig.csv	test.csv
Nodes	2485	1384
Positive edges	4969	100
Negative edges	-	900
Minimum degree	1	-
Maximum degree	167	-
75th percentile degree	5.00	-
Median degree	3.00	-

Table 1: Summary statistics of the dataset in the two files.

We are interested in the network feature of edge structure, particularly whether we can effectively predict the likelihood of a new edge's existence and which of them are most likely to exist. In summary, our research question is as follows:

Can we predict with a high degree of accuracy which 100 out of 1.000 edges in a graph are the most likely to appear?

The fundamental assumption underlying this approach is that the *positive* test edges can be considered as if they had been "removed" from an original graph that included both the training and test positive edges. Therefore, they have *something* in common with the original structure that makes them more similar to it than the *negative* edges, which are for example sampled from a random distribution.

3 Methods [link](#) · [index](#)

3.1 Dataset split

The methodology employed in this study is based on a well-established approach commonly utilized in machine learning methods. This process of division is crucial to ensure the impartial assessment of the various methods. Given the absence of labels in the test set, the training set was augmented and divided for this purpose. Four distinct splits were generated¹ based on the data presented in table 1:

1. **Baseline:** the ground truth for all methods because it contains only positive edges
2. **Training:** provides positive and negative edges to train the model (if needed)
3. **Evaluation:** provides positive and negative edges to compare the accuracy of the different methods
4. **Test:** the method with the best evaluation performance provides a predictions on this set

Set	Positive	Split %	Negative	Set	Positive	Split %	Negative
(training.csv)	4969	100%	0	(test.csv)	100	100%	900
Baseline	2484	50%	0				
Training	1491	30%	1491				
Evaluation	994	20%	994				

Table 2: Distribution of the original dataset across splits (left: training and evaluation, right: test)

A relevant observation from the table is that the training and evaluation sets are characterized by a different class ratio (1:1) compared to the test set (1:9). While this may lead to an optimistic expectation of downstream test accuracy, we considered important adhering to the best practice of training models with a balanced set of classes.

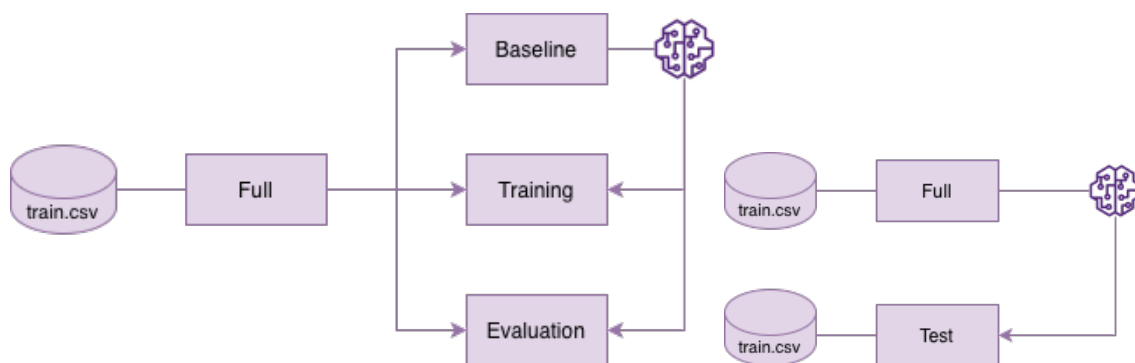


Figure 2: Overview of the dataset splits into training (left) and testing (right) sets.

3.2 Scoring method types

We are interested in estimating the likelihood of a set of edges belonging to a graph. This translates into **measuring the top-scoring edges with regard to the network's structure** where the more *similar* an edge is to the network structure, the more likely it is to exist. But what does *similar* mean in this context?

There exists a range of methodological approaches, each designed to address distinct definitions of such similarity:

¹All random processes, from dataset shuffling and splitting onward, used a fixed seed (42) to allow the reproducibility of all findings in this study.

1. **Neighborhood based:** the more common neighbors two nodes have, the more likely a new edge is to form between them
2. **Community based:** the more connected are communities the two nodes belong to, the more likely a new edge is to form between them
3. **Random-walk based:** the higher the probability of reaching one node from the other via a random walk, the more likely a new edge is to form between them
4. **Embedding based:** the closer two nodes are in a learned high-dimensional embedding space, the more likely a new edge is to form between them

Each of these methods provides an *encoding* of the nodes in the training network, allowing for the measurement of the similarity between a pair of them in candidate test edges. To this end, **three** distinct approaches were implemented, each belonging to a different family of methods.

3.3 Method 1: Common Neighbors (neighborhood-based)

This method defines the similarity between two nodes as the **number of neighbors they have in common**. In other words, the more neighbors two nodes have in common, the higher their score.

The similarity measure in this case is calculated as:

$$\sigma(x, y) = |N(x) \cap N(y)| \quad (1)$$

where (x, y) is a candidate edge and $N(X)$ indicates the set of neighbor nodes to the node x .

3.4 Method 2: Spectral Clustering (community-based)

This method defines the similarity between two nodes as the **distance between their embeddings**, which are obtained as the **eigenvectors of the graph Laplacian matrix**. In other words, nodes with similar spectral embeddings (i.e., those that lie in the same cluster or community) will receive a higher similarity score.

The similarity measure in this case is calculated as:

$$\sigma(x, y) = \frac{z_x^\top z_y}{\|z_x\| \|z_y\|}. \quad (2)$$

where (x, y) is a candidate edge and z_x indicates the spectral embedding of node x which appears as row of the $Z \in \mathbb{R}^{n \times d}$ embedding matrix. This matrix is obtained by computing the first d (non-trivial) eigenvectors of the normalized training graph Laplacian L . For example:

$$A \in \mathbb{R}^{n \times n}, \quad D = \text{diag}(d_1, \dots, d_n), \quad L = I - D^{-\frac{1}{2}} A D^{-\frac{1}{2}}, \quad Z = \begin{bmatrix} z_x \\ \vdots \\ z_y \end{bmatrix} \in \mathbb{R}^{n \times d} \quad (3)$$

3.5 Method 3: Graph Convolutional Network (embedding-based)

This method also defines the similarity between two nodes as the **distance between their embeddings**. These embeddings are obtained by propagating and transforming node features over the graph structure through

convolutional layers of a deep neural network. In other words, two nodes with GCN embeddings pointing in similar directions will receive a higher similarity score.

The similarity measure in this case is calculated as:

$$\sigma(x, y) = z_x^\top z_y \quad (4)$$

where (x, y) is a candidate edge and z_x indicates the GCN embedding of node x , which appears as a row of the embedding matrix $Z \in \mathbb{R}^{n \times d_{\text{emb}}}$. This matrix is obtained by applying two graph convolutional layers to a feature vector containing the **normalized degree** of each node.

Concretely, we first modify the adjacency matrix by adding self-loops and normalizing it:

$$\hat{A} = A + I, \quad d = \begin{bmatrix} d_1 & d_2 & \cdots & d_n \end{bmatrix}^\top, \quad \hat{D} = \text{diag}(d_1, \dots, d_n)^{-\frac{1}{2}}, \quad \tilde{A} = \hat{D} \hat{A} \hat{D} \quad (5)$$

Then we construct the input feature vector:

$$X = \frac{d - \mu}{\sigma + \varepsilon} \in \mathbb{R}^{n \times 1} \quad (6)$$

The first layer (with $W_1 \in \mathbb{R}^{1 \times H}$ and $b_1 \in \mathbb{R}^H$) maps the input feature vector to a hidden representation:

$$H^{(1)} = \text{Dropout}\left(\text{ReLU}(\tilde{A}(XW_1 + b_1))\right) \in \mathbb{R}^{n \times H} \quad (7)$$

where $\text{Dropout}(\dots)$ and $\text{ReLU}(\dots)$ respectively represent a *dropout layer* and the *rectified linear unit* activation. The ReLU nonlinearity allows the GCN to learn expressive linear node representations while dropout regularizes the model by randomly masking hidden units, reducing the risk of overfitting the (small) training set.

A second layer (with $W_2 \in \mathbb{R}^{H \times d_{\text{emb}}}$ and $b_2 \in \mathbb{R}^{d_{\text{emb}}}$) produces the final node embeddings:

$$H^{(2)} = Z = \tilde{A}(H^{(1)}W_2 + b_2) \in \mathbb{R}^{n \times d_{\text{emb}}} \quad (8)$$

This approach required a training phase, so we optimized the model iteratively on *mini-batches* of training edges. For each batch, containing node pairs and their binary labels, the model produced logits for all candidate edges. We then computed the binary cross-entropy loss with respect to the ground-truth labels and backpropagated it to obtain gradients. Finally, the **Adam** optimizer, with a learning rate of 10^{-4} , updated the parameters once per batch based on these gradients and its adaptive moment estimates.

4 Results [link](#) · [index](#)

4.1 Accuracy (method selection)

As previously mentioned in [3](#), the objective was to select the most suitable method based on the performance metrics derived from the evaluation set. To facilitate a quantitative comparison of the results obtained from different methods, a function was implemented that converts all top-scoring edge pairs as positive and the remaining as negative:

$$\begin{aligned} \mathbf{y}_{\text{pred}} &= [2.91, 0.27, 4.83, -0.12, 3.76] \rightarrow [1, 0, 1, 0, 1] \\ \mathbf{y}_{\text{val}} &= [1, 1, 1, 0, 0] \end{aligned} \quad (9)$$

We then compared these predictions with the ground truth values to obtain the accuracy score, which is the ratio of true positive edges to predicted positive edges:

$$\text{Accuracy}(\mathbf{y}_{\text{pred}}, \mathbf{y}_{\text{val}}) = \frac{1}{n} \sum_{i=1}^n \mathbb{I}(y_{\text{pred},i} = y_{\text{val},i}) \quad (10)$$

The accuracy measurement for each method, along with the corresponding evaluation set utilized for the calculation, is presented in the following:

Method	Accuracy	Trained on ¹	Evaluated on ¹
1. Common Neighbors (3.3)	0.9949	-	Evaluation
2. Spectral Clustering (3.4)	0.8933	-	Evaluation
3. Graph Convolutional Network (3.5)	0.7263	Training	Evaluation

Table 3: Accuracy measures of the three methods

4.2 Predictions (method 2)

To visualize the efficacy of the method in embedding the nodes in a space that accurately groups them according to their common clustering structure, we have plotted the first two non-trivial eigenvectors in the following diagram:

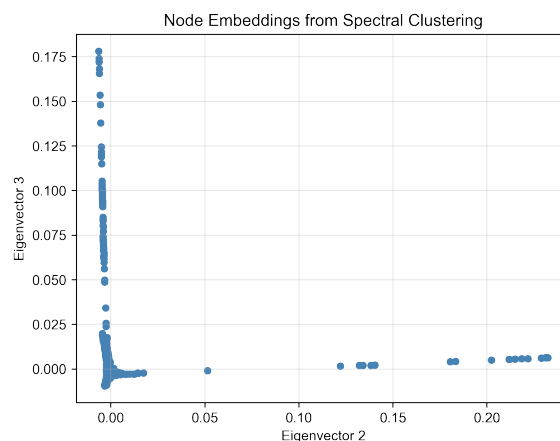


Figure 3: Subset of graph nodes in the 2-dimensional space generated from the first two non-trivial eigenvectors of the Laplacian matrix.

¹As seen in table [2](#)

Intuitively, it appears that the distances are incorporating different levels of clustering, interestingly rensabling the spectral representation illustrated in Figure 1.

We used the smallest $k = 10$ (non-trivial) eigenvectors to embed and compare the candidate edges. The following scoring edges from the testing set (with some of them omitted for brevity) resulted from method 2:

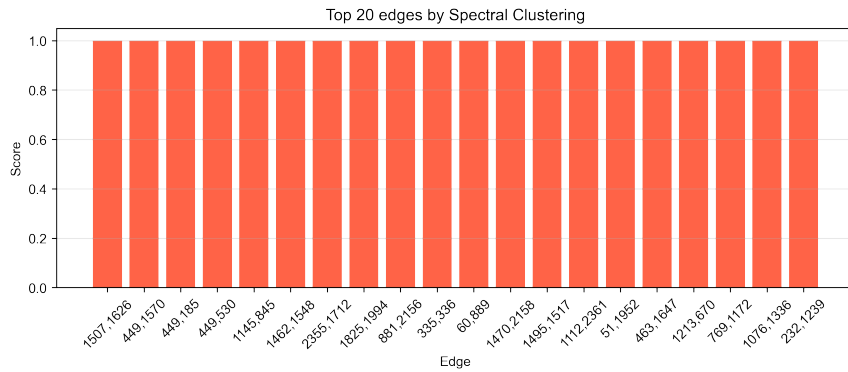


Figure 4: Visualization of the test set predictions for the top 20 scoring edges.

R	Edge	Score	R	Edge	Score	R	Edge	Score	R	Edge	Score
1	(1507, 1626)	1.0000	14	(1112, 2361)	0.9996	27	(881, 1488)	0.9992	40	(523, 2477)	0.9985
2	(449, 1570)	1.0000	15	(51, 1952)	0.9996	28	(479, 1660)	0.9992	41	(905, 1468)	0.9984
3	(449, 185)	0.9999	16	(463, 1647)	0.9995	29	(786, 2410)	0.9992	42	(505, 648)	0.9979
4	(449, 530)	0.9999	17	(1213, 670)	0.9995	30	(558, 614)	0.9992	43	(102, 1175)	0.9979
5	(1145, 845)	0.9999	18	(769, 1172)	0.9994	31	(102, 1760)	0.9992	44	(816, 1519)	0.9978
6	(1462, 1548)	0.9999	19	(1076, 1336)	0.9994	32	(505, 1547)	0.9992	45	(0, 842)	0.9978
7	(2355, 1712)	0.9999	20	(232, 1239)	0.9993	33	(505, 2462)	0.9992	46	(1541, 2289)	0.9971
8	(1825, 1994)	0.9998	21	(2439, 1402)	0.9993	34	(1706, 1708)	0.9989	47	(1639, 2463)	0.9970
9	(881, 2156)	0.9998	22	(1114, 1505)	0.9993	35	(921, 1355)	0.9989	48	(38, 1339)	0.9970
10	(335, 336)	0.9997	23	(1114, 1012)	0.9993	36	(463, 102)	0.9989	49	(769, 344)	0.9965
11	(60, 889)	0.9997	24	(1706, 1717)	0.9992	37	(316, 804)	0.9988	50	(1399, 486)	0.9961
12	(1470, 2158)	0.9997	25	(79, 1291)	0.9992	38	(178, 181)	0.9988	51	(490, 1346)	0.9961
13	(1495, 1517)	0.9997	26	(1579, 1075)	0.9992	39	(613, 1488)	0.9988	52	(103, 1189)	0.9959

Table 4: Test set predictions for the top 52 scoring edges.

In the interest of completeness, the same predictions were also provided for the other non-main methods in [A.2](#).

5 Discussion [link](#) · [index](#)

Despite the fact that Method 1 yielded the highest level of accuracy, it was not selected as the primary method, mainly due to its integer nature of its results. Consequently, the ranking system would have collapsed into an arbitrary sorting by label, compromising the ranking purpose. Furthermore, its outcome is exclusively dependent on the existence of common neighbors between the nodes of the candidate edges and the training set², a fact which was entirely unknown to us.

Method 2 was determined to be the optimal trade-off between **speed**, **explainability**, and **flexibility**. This approach enabled the calculation of real and reliable scoring values of the top edges. This might be due to the reasonable hypothesis that the nodes in the top ranking edges are part of a network area that already exhibits a high degree of clustering commonality, even though they may not have existing edges between each other.

Method 3 was considered the most intriguing, initially supposed to be leading to better results considering its different nature. The potential for enhancement, however, turned out to be greatly influenced by the capacity to define feature vectors with high semantic relevance. The adoption of a simple degree for each node resulted in the training loss reaching a plateau in the early stages of training (refer to Figure 5). This occurred despite the implementation of regularizations aimed at preventing overfitting, which can be attributed to the absence of more informative data concerning the dataset, such as more complex features at each node.

5.1 Success areas

The implementation of a robust **benchmarking** system 2 has been achieved, with careful consideration given to the different nature of the methods to ensure the comparability of their results (table 3). All methods demonstrated a **high extent of accuracy** overall. Nevertheless, method 2 was particularly appreciated for its balance between **accuracy**, **generalizability**, and **flexibility** 4.2. A further point of interest was the comparison of methods from **different families** 3, providing different insights into the network structure. Specifically, we had the opportunity to apply our **machine learning** background in the Graph Convolutional Method 3.5, where we obtained a promising learning curve (in Figure 5) and hands-on experience with its implementation.

5.2 Improvement areas

Despite the potential of the last method to surpass the first two in terms of performance, it relied on access to more **informative data** to generate substantial results. Additionally, the model could have **incorporated** other methods, such as a common neighbor estimate that would have been learned by the network.

With regard to method 2, a cross-validation with **hyperparameter tuning** could have been implemented to reliably predict the optimal number of eigenvectors to select for maximizing the accuracy.

An alternative method that could have been employed is the method of **enabling**. This indicates that a majority vote is employed on the top-ranking nodes, and multiple methods are weighted simultaneously to enhance overall performance and overcome the limitations associated with each method.

²Indeed, the method reported several same-scores A.2.1 and zero-scores for the top 100 scoring edges.

6 Conclusion [link](#) · [index](#)

We presented a reproducible, quantitative evaluation of various methods [3](#) for solving the link prediction problem [2](#), which can be applied to any graph structure. These methods can also be used to evaluate the relationship between nodes within the same network for various tasks and purposes. We reported the accuracy scores for all methods, as well as the predictions from the method we deemed optimal [4](#). Finally, we discussed the results and reflected on the study's strengths and limitations [5](#).

A Appendix [link](#) · [index](#)

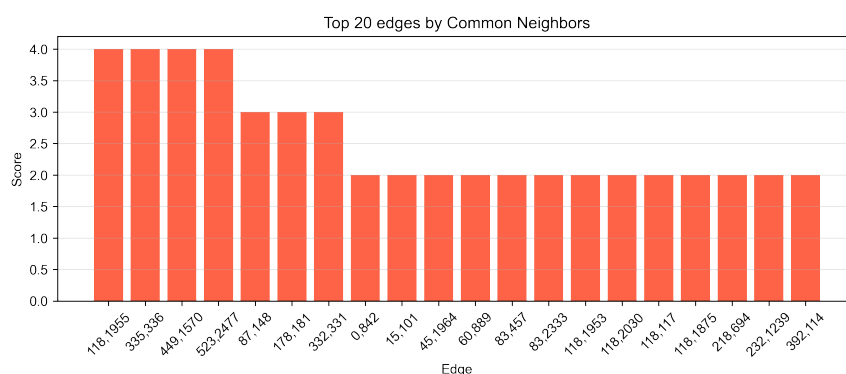
A.1 Sources

All sources and their respective histories, including but not limited to the findings and the facts presented in this study, the experiments conducted to understand the subject matter, as well as further details, are accessible to the public in this [GitHub repository](#).

A.2 Methods scoring on test set

Even though we considered method 2 as our main method to process the test set, for completeness here we provide the top- k predictions of the other methods presented in 3. For brevity, we include only a part of the 100 computer results.

A.2.1 Method 1: Common Neighbors (3.3)



R	Edge	R	Edge	R	Edge	R	Edge
1	(118, 1955)	4.0000	14	(118, 1953)	2.0000	27	(1145, 845)
2	(335, 336)	4.0000	15	(118, 2030)	2.0000	28	(1706, 1717)
3	(449, 1570)	4.0000	16	(118, 117)	2.0000	29	(46, 720)
4	(523, 2477)	4.0000	17	(118, 1875)	2.0000	30	(77, 837)
5	(87, 148)	3.0000	18	(218, 694)	2.0000	31	(85, 2009)
6	(178, 181)	3.0000	19	(232, 1239)	2.0000	32	(92, 166)
7	(332, 331)	3.0000	20	(392, 114)	2.0000	33	(240, 1363)
8	(0, 842)	2.0000	21	(449, 185)	2.0000	34	(323, 322)
9	(15, 101)	2.0000	22	(449, 530)	2.0000	35	(358, 2420)
10	(45, 1964)	2.0000	23	(490, 1346)	2.0000	36	(434, 1829)
11	(60, 889)	2.0000	24	(515, 901)	2.0000	37	(463, 102)
12	(83, 457)	2.0000	25	(1114, 1012)	2.0000	38	(477, 386)
13	(83, 2333)	2.0000	26	(1145, 845)	2.0000	39	(479, 1660)
						40	(498, 668)
						41	(505, 648)
						42	(515, 900)
						43	(543, 1467)
						44	(709, 710)
						45	(769, 1172)
						46	(786, 2410)
						47	(786, 1225)
						48	(881, 1488)
						49	(1112, 2361)
						50	(1213, 670)
						51	(1428, 1107)
						52	(1462, 1548)

A.2.2 Method 2: Spectral Clustering (3.4)

Refer to section 4.2.

A.2.3 Method 3: Graph Convolutional Network (3.5)

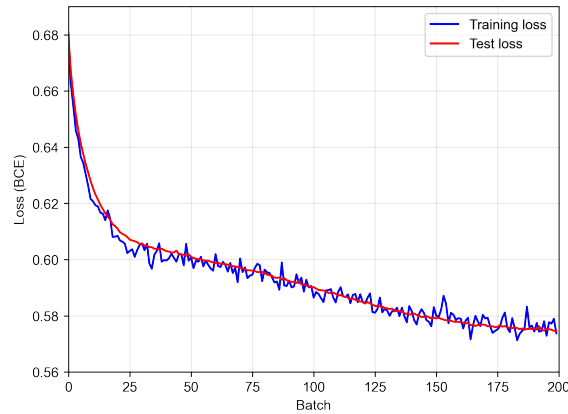
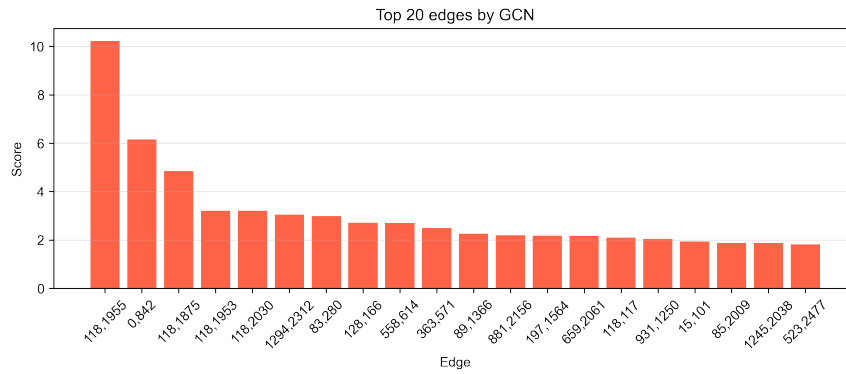


Figure 5: Training and evaluation loss of the GCN (before test scoring)



R	Edge	Score	R	Edge	Score	R	Edge	Score	R	Edge	Score
1	(118, 1955)	10.2221	14	(659, 2061)	2.1744	27	(1153, 761)	1.6179	40	(582, 1538)	1.2950
2	(0, 842)	6.1539	15	(118, 117)	2.1111	28	(643, 1114)	1.6155	41	(29, 1409)	1.2709
3	(118, 1875)	4.8476	16	(931, 1250)	2.0510	29	(816, 1519)	1.5807	42	(515, 901)	1.2690
4	(118, 1953)	3.2181	17	(15, 101)	1.9483	30	(352, 2439)	1.5441	43	(481, 963)	1.1770
5	(118, 2030)	3.2181	18	(85, 2009)	1.8815	31	(47, 2278)	1.4647	44	(180, 2064)	1.1742
6	(1294, 2312)	3.0610	19	(1245, 2038)	1.8799	32	(48, 1366)	1.4641	45	(1112, 2361)	1.1662
7	(83, 280)	2.9900	20	(523, 2477)	1.8242	33	(523, 690)	1.4309	46	(788, 2255)	1.1627
8	(128, 166)	2.7251	21	(218, 694)	1.7299	34	(971, 2092)	1.3928	47	(678, 2203)	1.1610
9	(558, 614)	2.7110	22	(196, 725)	1.7252	35	(1580, 1746)	1.3718	48	(700, 1453)	1.1559
10	(363, 571)	2.4953	23	(50, 1786)	1.7091	36	(1002, 1955)	1.3669	49	(319, 1538)	1.1373
11	(89, 1366)	2.2690	24	(144, 1932)	1.6582	37	(178, 181)	1.3409	50	(340, 547)	1.1157
12	(881, 2156)	2.2010	25	(116, 1959)	1.6228	38	(743, 1431)	1.3215	51	(392, 114)	1.1143
13	(197, 1564)	2.1861	26	(1153, 761)	1.6179	39	(505, 648)	1.3005	52	(92, 166)	1.1094

A.3 About Generative AI Usage

In the perspective of fair and ethical usage of LLMs, we state that the usage of GenAI has been limited to support tool for writing and debugging the source code and to correct grammatical mistakes in the text. In writing the present document, we felt the commitment to maintain the genuineness of our handwritten text as much as possible.

The following list presents examples GenAI model prompts as part of our workflow:

- **Proofreading:** "Is this paragraph grammatically correct? Is there a more cohesive way of putting it to words? (...)"
- **Clarification:** "Please explain the content of this slide. (...)"
- **Understanding:** "If method A and method B relate to the same category, why are their values differing so much? (...)"
- **Debugging:** "Why is this snippet taking such a long time to complete? Is there any problem? (...)"
- **Best practices:** "Is the way used in this snippet the most community-supported way of solving the problem? Are there any better ways? (...)"